Not many C/C++ programmers are aware of partial regex capabilities built into the C standard I/O library. Although `scanf/printf` are C-style I/O routines, they can still be used in C++ code. Many C++ programmers 'force' themselves to use `cin/cout` all the time even though it is sometimes not as flexible as `scanf/printf` and is also far slower.

In programming contests, especially ICPCs, coding time should *not* be the primary bottleneck. Once you figure out the 'worst AC algorithm' that will pass the given time limit, you are expected to be able to translate it into a bug-free code and fast!

Now, try some of the exercises below! If you need more than 10 lines of code to solve any of them, you should revisit and update your knowledge of your programming language(s)! A mastery of the programming languages you use and their built-in routines is extremely important and will help you a lot in programming contests.

Exercise 1.2.3: Produce working code that is *as concise as possible* for the following tasks:

1. Using **Java**, read in a double
   (e.g. `1.4732`, `15.324547327`, etc.)
   echo it, but with a minimum field width of 7 and 3 digits after the decimal point
   (e.g. `ss1.473` (where '`s`' denotes a space), `s15.325`, etc.)

2. Given an integer n ($n \leq 15$), print π to n digits after the decimal point (rounded).
   (e.g. for $n = 2$, print `3.14`; for $n = 4$, print `3.1416`; for $n = 5$, prints `3.14159`.)

3. Given a date, determine the day of the week (Monday, ..., Sunday) on that day.
   (e.g. 9 August 2010—the launch date of the first edition of this book—is a Monday.)

4. Given n random integers, print the distinct (unique) integers in sorted order.

5. Given the distinct and valid birthdates of n people as triples (DD, MM, YYYY), order them first by ascending birth months (MM), then by ascending birth dates (DD), and finally by ascending age.

6. Given a list of *sorted* integers L of size up to $1M$ items, determine whether a value v exists in L with no more than 20 comparisons (more details in Section 2.2).

7. Generate all possible permutations of {'A', 'B', 'C', ..., 'J'}, the first $N = 10$ letters in the alphabet (see Section 3.2.1).

8. Generate all possible subsets of {0, 1, 2, ..., N-1}, for $N = 20$ (see Section 3.2.1).

9. Given a string that represents a base X number, convert it to an equivalent string in base Y, $2 \leq X, Y \leq 36$. For example: "FF" in base X = 16 (hexadecimal) is "255" in base $Y_1 = 10$ (decimal) and "11111111" in base $Y_2 = 2$ (binary). See Section 5.3.2.

10. Let's define a 'special word' as a lowercase alphabet followed by two consecutive digits. Given a string, replace all 'special words' of length 3 with 3 stars "***", e.g.
    S = "line: a70 and z72 will be replaced, aa24 and a872 will not"
    should be transformed into:
    S = "line: *** and *** will be replaced, aa24 and a872 will not".

11. Given a *valid* mathematical expression involving '+', '-', '*', '/', '(', and ')' in a single line, evaluate that expression. (e.g. a rather complicated but valid expression `3 + (8 - 7.5) * 10 / 5 - (2 + 5 * 7)` should produce `-33.0` when evaluated with standard operator precedence.)

```
// C++ code for task 6, assuming all necessary includes have been done
int main() {
  int n = 5, L[] = {10, 7, 5, 20, 8}, v = 7;
  sort(L, L + n);
  printf("%d\n", binary_search(L, L + n, v));
}

// C++ code for task 7, assuming all necessary includes have been done
int main() {
  int p[10], N = 10; for (int i = 0; i < N; i++) p[i] = i;
  do {
    for (int i = 0; i < N; i++) printf("%c ", 'A' + p[i]);
    printf("\n");
  }
  while (next_permutation(p, p + N));
}

// C++ code for task 8, assuming all necessary includes have been done
int main() {
  int p[20], N = 20;
  for (int i = 0; i < N; i++) p[i] = i;
  for (int i = 0; i < (1 << N); i++) {
    for (int j = 0; j < N; j++)
      if (i & (1 << j))                                  // if bit j is on
        printf("%d ", p[j]);                          // this is part of set
    printf("\n");
} }

// Java code for task 9, assuming all necessary imports have been done
class Main {
  public static void main(String[] args) {
    String str = "FF"; int X = 16, Y = 10;
    System.out.println(new BigInteger(str, X).toString(Y));
} }

// Java code for task 10, assuming all necessary imports have been done
class Main {
  public static void main(String[] args) {
    String S = "line: a70 and z72 will be replaced, aa24 and a872 will not";
    System.out.println(S.replaceAll("(^| )+[a-z][0-9][0-9]( |$)+", " *** "));
} }

// Java code for task 11, assuming all necessary imports have been done
class Main {
  public static void main(String[] args) throws Exception {
    ScriptEngineManager mgr = new ScriptEngineManager();
    ScriptEngine engine = mgr.getEngineByName("JavaScript");     // "cheat"
    Scanner sc = new Scanner(System.in);
    while (sc.hasNextLine()) System.out.println(engine.eval(sc.nextLine()));
} }
```

There are generally three common methods to search for an item in an array:

1. $O(n)$ Linear Search: Consider every element from index 0 to index $n-1$ (avoid this whenever possible).
2. $O(\log n)$ Binary Search: Use `lower_bound`, `upper_bound`, or `binary_search` in C++ STL `algorithm` (or Java `Collections.binarySearch`). If the input array is unsorted, it is necessary to sort the array at least once (using one of the $O(n \log n)$ sorting algorithm above) before executing one (or *many*) Binary Search(es).
3. $O(1)$ with Hashing: This is a useful technique to use when fast access to known values are required. If a suitable hash function is selected, the probability of a collision to be made is negligibly small. Still, this technique is rarely used and we can live without it[6] for most (contest) problems.

Visualization: `www.comp.nus.edu.sg/~stevenha/visualization/sorting.html`

Source code: `ch2_02_algorithm_collections.cpp/java`

- Array of Booleans: C++ STL `bitset` (Java `BitSet`)
  If our array needs only to contain Boolean values (1/true and 0/false), we can use an alternative data structure other than an array—a C++ STL `bitset`. The `bitset` supports useful operations like `reset()`, `set()`, the `[]` operator and `test()`.

Source code: `ch5_06_primes.cpp/java`, also see Section 5.5.1

- Bitmasks a.k.a. lightweight, small sets of Booleans (native support in C/C++/Java)
  An integer is stored in a computer's memory as a sequence/string of bits. Thus, we can use integers to represent a *lightweight* small set of Boolean values. All set operations then involve only the bitwise manipulation of the corresponding integer, which makes it a *much more efficient* choice when compared with the C++ STL `vector<bool>`, `bitset`, or `set<int>` options. Such speed is important in competitive programming. *Some* important operations that are used in this book are shown below.

```
Message: Check if j-th bit (from right) of S is on
                                {F D B }(set)
S=42 (dec)                    = 101010 (bin)
j=3, 1<<j=8 (dec)             = 001000 (bin)
                                ------ AND
T=8 (dec) =                     001000 (bin)
                                {  D   }(set)
S = 42 (set all n = - bits) | LSOne set check clear toggle | j = 3 shift left shift right
```

Figure 2.1: Bitmask Visualization

1. Representation: A 32 (or 64)-bit *signed* integer for up to 32 (or 64) items[7]. Without a loss of generality, all examples below use a 32-bit signed integer called S.

[6] However, questions about hashing frequently appear in interviews for IT jobs.

[7] To avoid issues with the two's complement representation, use a 32-bit/64-bit *signed* integer to represent bitmasks of up to 30/62 items only, respectively.

```
Example:           5| 4| 3| 2| 1| 0 <- 0-based indexing from right
                  32|16| 8| 4| 2| 1 <- power of 2
S = 34 (base 10) = 1| 0| 0| 0| 1| 0 (base 2)
                   F| E| D| C| B| A <- alternative alphabet label
```

In the example above, the integer $S = 34$ or `100010` in binary also represents a small set $\{1, 5\}$ with a 0-based indexing scheme in increasing digit significance (or {B, F} using the alternative alphabet label) because the second and the sixth bits (counting from the right) of S are on.

2. To multiply/divide an integer by 2, we only need to shift the bits in the integer left/right, respectively. This operation (especially the shift left operation) is important for the next few examples below. Notice that the truncation in the shift right operation automatically rounds the division-by-2 down, e.g. $17/2 = 8$.

```
S                  = 34 (base 10) =  100010 (base 2)
S = S << 1 = S * 2 = 68 (base 10) = 1000100 (base 2)
S = S >> 2 = S / 4 = 17 (base 10) =   10001 (base 2)
S = S >> 1 = S / 2 =  8 (base 10) =    1000 (base 2) <- LSB is gone
                                     (LSB = Least Significant Bit)
```

3. To set/turn on the `j`-th item (0-based indexing) of the set,
   use the bitwise OR operation `S |= (1 << j)`.

```
S = 34 (base 10) = 100010 (base 2)
j = 3, 1 << j    = 001000 <- bit '1' is shifted to the left 3 times
                  -------- OR (true if either of the bits is true)
S = 42 (base 10) = 101010 (base 2) // update S to this new value 42
```

4. To check if the `j`-th item of the set is on,
   use the bitwise AND operation `T = S & (1 << j)`.
   If `T = 0`, then the `j`-th item of the set is off.
   If `T != 0` (to be precise, `T = (1 << j)`), then the `j`-th item of the set is on.
   See Figure 2.1 for one such example.

```
S = 42 (base 10) = 101010 (base 2)
j = 3, 1 << j    = 001000 <- bit '1' is shifted to the left 3 times
                  -------- AND (only true if both bits are true)
T = 8 (base 10)  = 001000 (base 2) -> not zero, the 3rd item is on

S = 42 (base 10) = 101010 (base 2)
j = 2, 1 << j    = 000100 <- bit '1' is shifted to the left 2 times
                  -------- AND
T = 0 (base 10)  = 000000 (base 2) -> zero, the 2rd item is off
```

5. To clear/turn off the `j`-th item of the set,
   use[8] the bitwise AND operation `S &= ~(1 << j)`.

```
S = 42 (base 10) = 101010 (base 2)
j = 1, ~(1 << j) = 111101 <- '~' is the bitwise NOT operation
                  -------- AND
S = 40 (base 10) = 101000 (base 2) // update S to this new value 40
```

[8] Use brackets a lot when doing bit manipulation to avoid accidental bugs due to operator precedence.

6. To toggle (flip the status of) the `j`-th item of the set,
   use the bitwise XOR operation `S ^= (1 << j)`.

```
S = 40 (base 10) = 101000 (base 2)
j = 2, (1 << j)  = 000100 <- bit ‘1’ is shifted to the left 2 times
                   -------- XOR <- true if both bits are different
S = 44 (base 10) = 101100 (base 2) // update S to this new value 44

S = 40 (base 10) = 101000 (base 2)
j = 3, (1 << j)  = 001000 <- bit ‘1’ is shifted to the left 3 times
                   -------- XOR <- true if both bits are different
S = 32 (base 10) = 100000 (base 2) // update S to this new value 32
```

7. To get the value of the least significant bit that is on (first from the right),
   use `T = (S & (-S))`.

```
 S   =  40 (base 10) =  000...000101000 (32 bits, base 2)
-S   = -40 (base 10) =  111...111011000 (two’s complement)
                       ----------------- AND
 T   =   8 (base 10) =  000...000001000 (3rd bit from right is on)
```

8. To turn on *all* bits in a set of size n, use `S = (1 << n) - 1`
   (be careful with overflows).

```
Example for n = 3
S + 1 = 8 (base 10) = 1000 <- bit ‘1’ is shifted to left 3 times
                         1
                     ------ -
S     = 7 (base 10) =  111 (base 2)

Example for n = 5
S + 1 = 32 (base 10) = 100000 <- bit ‘1’ is shifted to left 5 times
                            1
                      -------- -
S     = 31 (base 10) =  11111 (base 2)
```

Visualization: `www.comp.nus.edu.sg/~stevenha/visualization/bitmask.html`

Source code: `ch2_03_bit_manipulation.cpp/java`

Many bit manipulation operations are written as preprocessor macros in our C/C++ example source code (but written plainly in our Java example code since Java does not support macros).

- Linked List: C++ STL `list` (Java `LinkedList`)
  Although this data structure almost always appears in data structure and algorithm textbooks, the Linked List is usually avoided in typical (contest) problems. This is due to the inefficiency in accessing elements (a linear scan has to be performed from the head or the tail of a list) and the usage of pointers makes it prone to runtime errors if not implemented properly. In this book, almost all forms of Linked List have been replaced by the more flexible C++ STL `vector` (Java `Vector`).

The only exception is probably UVa 11988 - Broken Keyboard (a.k.a. Beiju Text)—where you are required to dynamically maintain a (linked) list of characters and efficiently insert a new character *anywhere* in the list, i.e. at front (head), current, or back (tail) of the (linked) list. Out of 1903 UVa problems that the authors have solved, this is likely to be the only pure linked list problem we have encountered thus far.

- Stack: C++ STL `stack` (Java `Stack`)
  This data structure is often used as part of algorithms that solve certain problems (e.g. bracket matching in Section 9.4, Postfix calculator and Infix to Postfix conversion in Section 9.27, finding Strongly Connected Components in Section 4.2.9 and Graham's scan in Section 7.3.7). A stack only allows for $O(1)$ insertion (push) and $O(1)$ deletion (pop) from the top. This behavior is usually referred to as Last In First Out (LIFO) and is reminiscent of literal stacks in the real world. Typical C++ STL `stack` operations include `push()`/`pop()` (insert/remove from top of stack), `top()` (obtain content from the top of stack), and `empty()`.

- Queue: C++ STL `queue` (Java `Queue`[9])
  This data structure is used in algorithms like Breadth First Search (BFS) in Section 4.2.2. A queue only allows for $O(1)$ insertion (enqueue) from the back (tail) and $O(1)$ deletion (dequeue) from the front (head). This behavior is similarly referred to as First In First Out (FIFO), just like actual queues in the real world. Typical C++ STL `queue` operations include `push()`/`pop()` (insert from back/remove from front of queue), `front()`/`back()` (obtain content from the front/back of queue), and `empty()`.

- Double-ended Queue (Deque): C++ STL `deque` (Java `Deque`[10])
  This data structure is very similar to the resizeable array (vector) and queue above, except that deques support fast $O(1)$ insertions and deletions at *both* the beginning and the end of the deque. This feature is important in certain algorithm, e.g. the Sliding Window algorithm in Section 9.31. Typical C++ STL `deque` operations include `push_back()`, `pop_front()` (just like the normal queue), `push_front()` and `pop_back()` (specific for deque).

Visualization: `www.comp.nus.edu.sg/~stevenha/visualization/list.html`

Source code: `ch2_04_stack_queue.cpp/java`

Exercise 2.2.1*: Suppose you are given an *unsorted* array S of n integers. Solve each of the following tasks below with the best possible algorithms that you can think of and analyze their time complexities. Let's assume the following constraints: $1 \leq n \leq 100K$ so that $O(n^2)$ solutions are theoretically infeasible in a contest environment.

1. Determine if S contains one or more pairs of duplicate integers.

2*. Given an integer v, find two integers $a, b \in S$ such that $a + b = v$.

3*. Follow-up to Question 2: what if the given array S is *already sorted*?

4*. Print the integers in S that fall between a range $[a \ldots b]$ (inclusive) in sorted order.

5*. Determine the length of the longest increasing *contiguous* sub-array in S.

6. Determine the median (50th percentile) of S. Assume that n is odd.

[9] The Java `Queue` is only an *interface* that is usually instantiated with Java `LinkedList`.
[10] The Java `Deque` is also an *interface*. `Deque` is usually instantiated with Java `LinkedList`.

Exercise 2.2.2: There are several other 'cool' tricks possible with bit manipulation techniques but these are rarely used. Please implement these tasks with bit manipulation:

1. Obtain the remainder (modulo) of S when it is divided by N (N is a power of 2)
   e.g. $S = (7)_{10}$ % $(4)_{10} = (111)_2$ % $(100)_2 = (11)_2 = (3)_{10}$.
2. Determine if S is a power of 2.
   e.g. $S = (7)_{10} = (111)_2$ is not a power of 2, but $(8)_{10} = (100)_2$ is a power of 2.
3. Turn off the last bit in S, e.g. $S = (40)_{10} = (10\underline{1}000)_2 \rightarrow S = (32)_{10} = (10\underline{0}000)_2$.
4. Turn on the last zero in S, e.g. $S = (41)_{10} = (10100\underline{0}1)_2 \rightarrow S = (43)_{10} = (10101\underline{1}1)_2$.
5. Turn off the last consecutive run of ones in S
   e.g. $S = (39)_{10} = (100\underline{111})_2 \rightarrow S = (32)_{10} = (100\underline{000})_2$.
6. Turn on the last consecutive run of zeroes in S
   e.g. $S = (36)_{10} = (1001\underline{00})_2 \rightarrow S = (39)_{10} = (1001\underline{11})_2$.

7*. Solve UVa 11173 - Grey Codes with a *one-liner* bit manipulation expression for each test case, i.e. find the k-th Gray code.

8*. Let's reverse the UVa 11173 problem above. Given a gray code, find its position k using bit manipulation.

Exercise 2.2.3*: We can also use a *resizeable* array (C++ STL `vector` or Java `Vector`) to implement an efficient stack. Figure out how to achieve this. Follow up question: Can we use a *static* array, linked list, or deque instead? Why or why not?

Exercise 2.2.4*: We can use a linked list (C++ STL `list` or Java `LinkedList`) to implement an efficient queue (or deque). Figure out how to achieve this. Follow up question: Can we use a resizeable array instead? Why or why not?

Programming exercises involving linear data structures (and algorithms) with libraries:

- 1D Array Manipulation, e.g. array, C++ STL `vector` (or Java `Vector`/`ArrayList`)
  1. *UVa 00230 - Borrowers* (a bit of string parsing, see Section 6.2; maintain list of sorted books; sort key: author names first and if ties, by title; the input size is small although not stated; we do not need to use balanced BST)
  2. UVa 00394 - Mapmaker (any n-dimensional array is stored in computer memory as a single dimensional array; follow the problem description)
  3. UVa 00414 - Machined Surfaces (get longest stretch of 'B's)
  4. UVa 00467 - Synching Signals (linear scan, 1D boolean flag)
  5. UVa 00482 - Permutation Arrays (you may need to use a string tokenizer—see Section 6.2—as the size of the array is not specified)
  6. UVa 00591 - Box of Bricks (sum all items; get the average; sum the total absolute differences of each item from the average divided by two)
  7. *UVa 00665 - False Coin* (use 1D boolean flags; all coins are initially potential false coins; if '=', all coins on the left and right are not false coins; if '<' or '>', all coins not on the left and right are not false coins; check if there is only one candidate false coin left at the end)
  8. UVa 00755 - 487-3279 (Direct Addressing Table; convert the letters except Q & Z to 2-9; keep '0'-'9' as 0-9; sort the integers; find duplicates if any)

9. **UVa 10038 - Jolly Jumpers *** (use 1D boolean flags to check $[1..n-1]$)
10. UVa 10050 - Hartals (1D boolean flag)
11. UVa 10260 - Soundex (Direct Addressing Table for soundex code mapping)
12. UVa 10978 - Let's Play Magic (1D string array manipulation)
13. *UVa 11093 - Just Finish it up* (linear scan, circular array, a bit challenging)
14. UVa 11192 - Group Reverse (character array)
15. UVa 11222 - Only I did it (use several 1D arrays to simplify this problem)
16. **UVa 11340 - Newspaper *** (DAT; see Hashing in Section 2.3)
17. UVa 11496 - Musical Loop (store data in 1D array, count the peaks)
18. UVa 11608 - No Problem (use three arrays: created; required; available)
19. UVa 11850 - Alaska (for each integer location from 0 to 1322; can Brenda reach (anywhere within 200 miles of) any charging stations?)
20. *UVa 12150 - Pole Position* (simple manipulation)
21. ***UVa 12356 - Army Buddies **** (similar to deletion in doubly linked lists, but we can still use a 1D array for the underlying data structure)

- 2D Array Manipulation
  1. UVa 00101 - The Blocks Problem ('stack' like simulation; but we need to access the content of each stack too, so it is better to use 2D array)
  2. UVa 00434 - Matty's Blocks (a kind of visibility problem in geometry, solvable with using 2D array manipulation)
  3. UVa 00466 - Mirror Mirror (core functions: rotate and reflect)
  4. UVa 00541 - Error Correction (count the number of '1's for each row/col; all of them must be even; if $\exists$ an error, check if it is on the same row and col)
  5. UVa 10016 - Flip-flop the Squarelotron (tedious)
  6. UVa 10703 - Free spots (use 2D boolean array of size 500×500)
  7. **UVa 10855 - Rotated squares *** (string array, 90° clockwise rotation)
  8. **UVa 10920 - Spiral Tap *** (simulate the process)
  9. UVa 11040 - Add bricks in the wall (non trivial 2D array manipulation)
  10. UVa 11349 - Symmetric Matrix (use long long to avoid issues)
  11. UVa 11360 - Have Fun with Matrices (do as asked)
  12. **UVa 11581 - Grid Successors *** (simulate the process)
  13. UVa 11835 - Formula 1 (do as asked)
  14. *UVa 12187 - Brothers* (simulate the process)
  15. *UVa 12291 - Polyomino Composer* (do as asked, a bit tedious)
  16. *UVa 12398 - NumPuzz I* (simulate backwards, do not forget to mod 10)
- C++ STL `algorithm` (Java `Collections`)
  1. UVa 00123 - Searching Quickly (modified comparison function, use `sort`)
  2. **UVa 00146 - ID Codes *** (use `next_permutation`)
  3. UVa 00400 - Unix ls (this command very frequently used in UNIX)
  4. UVa 00450 - Little Black Book (tedious sorting problem)
  5. *UVa 00790 - Head Judge Headache* (similar to UVa 10258)
  6. UVa 00855 - Lunch in Grid City (sort, median)
  7. UVa 01209 - Wordfish (LA 3173, Manila06) (STL `next` and `prev_permutation`)
  8. *UVa 10057 - A mid-summer night ...* (involves the median, use STL `sort`, `upper_bound`, `lower_bound` and some checks)

9. **UVa 10107 - What is the Median? *** (find median of a *growing*/dynamic list of integers; still solvable with multiple calls of `nth_element` in `algorithm`)
10. UVa 10194 - Football a.k.a. Soccer (multi-fields sorting, use `sort`)
11. **UVa 10258 - Contest Scoreboard *** (multi-fields sorting, use `sort`)
12. *UVa 10698 - Football Sort* (multi-fields sorting, use `sort`)
13. UVa 10880 - Colin and Ryan (use `sort`)
14. UVa 10905 - Children's Game (modified comparison function, use `sort`)
15. UVa 11039 - Building Designing (use `sort` then count different signs)
16. UVa 11321 - Sort Sort and Sort (be careful with negative mod!)
17. UVa 11588 - Image Coding (`sort` simplifies the problem)
18. UVa 11777 - Automate the Grades (`sort` simplifies the problem)
19. UVa 11824 - A Minimum Land Price (`sort` simplifies the problem)
20. *UVa 12541 - Birthdates* (LA6148, HatYai12, `sort`, pick youngest and oldest)

- Bit Manipulation (both C++ STL `bitset` (Java `BitSet`) and bitmask)
  1. UVa 00594 - One Little, Two Little ... (manipulate bit string with `bitset`)
  2. UVa 00700 - Date Bugs (can be solved with `bitset`)
  3. UVa 01241 - Jollybee Tournament (LA 4147, Jakarta08, easy)
  4. ***UVa 10264 - The Most Potent Corner **** (heavy bitmask manipulation)
  5. *UVa 11173 - Grey Codes* (D & C pattern or one liner bit manipulation)
  6. UVa 11760 - Brother Arif, ... (separate row+col checks; use two bitsets)
  7. **UVa 11926 - Multitasking *** (use 1M `bitset` to check if a slot is free)
  8. **UVa 11933 - Splitting Numbers *** (an exercise for bit manipulation)
  9. IOI 2011 - Pigeons (this problem becomes simpler with bit manipulation but the final solution requires much more than that.)
- C++ STL `list` (Java `LinkedList`)
  1. **UVa 11988 - Broken Keyboard ... *** (rare linked list problem)
- C++ STL `stack` (Java `Stack`)
  1. UVa 00127 - "Accordian" Patience (shuffling `stack`)
  2. **UVa 00514 - Rails *** (use `stack` to simulate the process)
  3. **UVa 00732 - Anagram by Stack *** (use `stack` to simulate the process)
  4. ***UVa 01062 - Containers **** (LA 3752, WorldFinals Tokyo07, simulation with `stack`; maximum answer is 26 stacks; $O(n)$ solution exists)
  5. UVa 10858 - Unique Factorization (use `stack` to help solving this problem)
     Also see: implicit `stack`s in recursive function calls and Postfix conversion/evaluation in Section 9.27.
- C++ STL `queue` and `deque` (Java `Queue` and `Deque`)
  1. UVa 00540 - Team Queue (modified 'queue')
  2. **UVa 10172 - The Lonesome Cargo ... *** (use both `queue` and `stack`)
  3. **UVa 10901 - Ferry Loading III *** (simulation with `queue`)
  4. UVa 10935 - Throwing cards away I (simulation with `queue`)
  5. **UVa 11034 - Ferry Loading IV *** (simulation with `queue`)
  6. *UVa 12100 - Printer Queue* (simulation with `queue`)
  7. *UVa 12207 - This is Your Queue* (use both `queue` and `deque`)
     Also see: `queue`s in BFS (see Section 4.2.2)